Musanze Harvest Intelligence Service

SWE 3513 Artificial Intelligence - Individual Assignment 2 - Technical Report (INES Ruhengeri)

Full name	Aisha Niyonsaba
Registration number	25/27104 (REGDIGITS 2527104)
Institutional username	ug2527104@ines.ac.rw
Seed (LLLL)	7104
Verification code	**AI-A2-7104**
Dataset fingerprint (SHA-256)	`73f47f269966ba51893b7732cb9cf36d181464529af694db9b4b792cf2514924`

1. Identity and decision problem

Stakeholder. The field officer of a potato cooperative in Musanze, who receives farm records in inconsistent spreadsheets and needs an offline, evidence-based tool. A simple API allows later integration with a mobile application.

Prediction questions.

Task	Question for the field officer	Target
Regression	How many tons will this farm harvest?	yield_tons
Classification	Is this farm at high production risk (yield per hectare below the median)?	high_risk (0/1)
Clustering	Which farms have similar growing conditions and can be visited or advised together?	none (unsupervised)

Ethical boundary. The system is a classroom decision-support prototype trained on **synthetic** data. It does not replace an agronomist and must not be used to decide access to finance, insurance or public services. Every prediction needs human review. Inputs are rejected when they fall outside the ranges the model was trained on.

System overview. One command (`python run_all.py --student-id ug2527104@ines.ac.rw --seed 7104`) generates the dataset, validates and cleans it, trains and evaluates three kinds of machine learning, saves all artifacts and models, and the same saved pipelines are used by `predict.py` and the FastAPI service.

2. Data and reproducibility

Column	Generation rule	Role
rainfall_mm	Uniform 600-1400	numeric feature
avg_temp_c	Uniform 12-26	numeric feature
soil_moisture_pct	Uniform 20-80	numeric feature
fertilizer_kg	Uniform 0-250	numeric feature
field_size_ha	Uniform 0.2-4.0	numeric feature
season	Random A or B	categorical feature
cooperative_support	Random 0 or 1	binary feature
yield_tons	0.004 rain + 0.06 soil + 0.02 fert + 1.30 size - 0.25 |temp-18| + 0.70 support + season effect (+0.50 A, -0.20 B) + N(0, 0.80), clipped at 0.50	regression target
high_risk	1 if yield_tons / field_size_ha is below its median	classification target

Personal seed rule. The seed is LLLL = 7104 (last four digits of 25/27104) and the generator uses `numpy.random.default_rng(7104)`. Base rows = 240 + (7104 mod 41) = 240 + 11 = **251**. The number of deliberate missing values is m = round(0.05 x 251) = **13** in soil_moisture_pct and 13 in fertilizer_kg (different random positions); then 3 exact duplicates are appended, giving **254** raw rows. `run_all.py` rejects any other seed.

Data-quality evidence (artifacts/data_quality_report.json)

Check	Result
Raw rows	254
Exact duplicate rows found / removed	3 / 3
Missing values in raw CSV	soil_moisture_pct = 15, fertilizer_kg = 14 (13 + 13 planted; the 3 duplicated rows copied 2 + 1 of them)
Missing values after de-duplication	soil_moisture_pct = 13, fertilizer_kg = 13
Rows after cleaning	251
Share of high_risk = 1	0.498
Checks passed	schema, ranges, categories, targets_not_missing

Handling

Duplicates are dropped (first occurrence kept). Missing values are **not** filled on the whole dataset; they are median-imputed inside the pipeline, so the medians come from the training split only. The CSV is written with four decimals and LF line endings, so its bytes (and SHA-256) are the same on any operating system.

SHA-256 fingerprint (artifacts/data_fingerprint.txt)

`73f47f269966ba51893b7732cb9cf36d181464529af694db9b4b792cf2514924`

Re-running run_all.py twice reproduces this fingerprint and the metrics exactly (verified by an automated test). To recreate the CSV alone: `python data/generate_data.py --student-id ug2527104@ines.ac.rw --seed 7104`.

3. Pipeline design

Train/test separation. After cleaning (251 rows) the data is split once with `train_test_split(test_size=0.25, random_state=7104, stratify=high_risk)` into **188 training** and **63 test** farms. The same split is used for all three tasks. The test set is touched only for final reporting.

Preprocessing flow (one ColumnTransformer inside every Pipeline).

Feature group	Columns	Transformation (fit on training rows only)
Numeric	rainfall_mm, avg_temp_c, soil_moisture_pct, fertilizer_kg, field_size_ha	SimpleImputer(median) then StandardScaler
Categorical	season	OneHotEncoder(drop='if_binary', handle_unknown='error')
Binary	cooperative_support	passthrough (0/1)

Leakage controls. yield_tons, yield per hectare and high_risk are never model inputs (`assert_no_leakage` runs when the preprocessor is built, and the API schema rejects extra fields such as yield_tons). high_risk is derived from the yield, so using it (or yield) as a feature would hide the real difficulty of the task. Imputers, scalers and encoders are never fitted on the full dataset. Clustering uses only the seven input features.

Model candidates and selection rule.

Task	Baseline	Candidates	Selection (training data only)
Regression	Mean predictor	Linear regression, Random forest (200 trees), Gradient boosting	Lowest 5-fold CV RMSE
Classification	Majority class	Logistic regression, Random forest (200 trees), Gradient boosting	Highest 5-fold CV F1
Clustering	-	KMeans, K = 2..6 (n_init=10)	Highest silhouette score

All random states equal 7104. Final models are the selected pipelines fitted on the training split, so the reported test metrics describe exactly the saved models (`models/*.joblib`). The same pipeline objects are loaded by predict.py and app.py, so training and inference apply identical transformations.

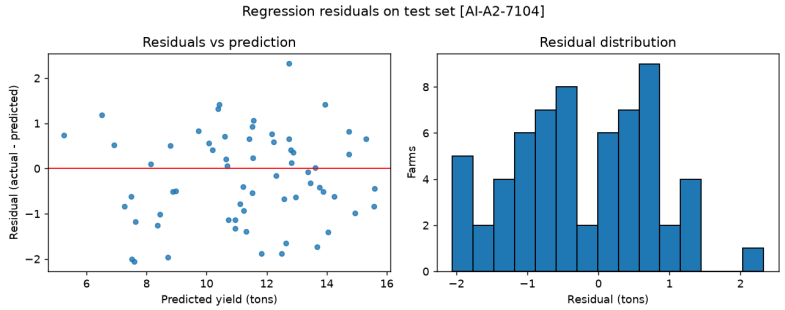
Code structure. src/config.py (constants, identity), validation.py (seed, schema, ranges, leakage), preprocessing.py, modeling.py, clustering.py, evaluation.py (plots, predictions), schemas.py (typed input), inference.py (artifact loading, prediction), pipeline.py (orchestration); entry points run_all.py, predict.py, app.py.

4. Regression and classification

Regression (63 held-out test farms)

Model	CV RMSE (train)	Test MAE	Test RMSE	Test R2
baseline_mean	2.6678	2.2312	2.8161	-0.0857
linear_regression **(selected)**	0.9706	0.852	1.0163	0.8586
random_forest	1.2693	1.2464	1.499	0.6924
gradient_boosting	1.1328	1.0842	1.309	0.7654

Selection. linear_regression has the lowest 5-fold cross-validated RMSE on the training split (0.9706 vs 2.6678 for the mean baseline). Selection used training-only cross-validation, not the test set. Linear regression wins because the true yield formula is almost linear; flexible tree models need more than 188 rows to match it. The mean baseline has R2 below zero on the test set, so the selected model is far better than guessing the average.

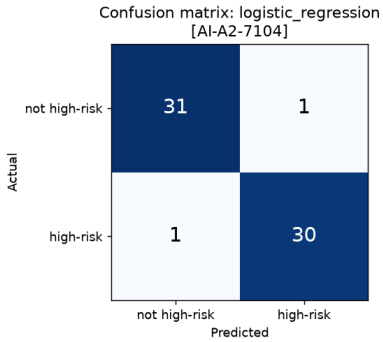


Residual interpretation and one important error (my analysis). Residuals are centred on zero with no strong curve against the prediction, so the linear model is not systematically biased. The largest regression error is farm row 244. The actual yield was 15.07 t but the model predicted 12.75 t, so it under-predicted by 2.32 t. This farm has low rainfall (about 702 mm) but a large field (3.45 ha), and its soil moisture value was missing, so the pipeline filled it with the training median. I see two possible causes. First, the data has random noise with standard deviation 0.80 t, and an error of 2.32 t is about three times that, so this farm was probably just lucky. Second, if the real soil moisture was higher than the median, the imputed value made the prediction too low. The model itself is not biased, because most residuals are small and centred around zero.

Classification (positive class = high_risk)

Model	CV F1	Precision	Recall	F1	ROC-AUC	Confusion [[TN,FP],[FN,TP]]
baseline_majority	0.1309	0.0	0.0	0.0	0.5	[[32, 0], [31, 0]]
logistic_regression **(selected)**	0.9339	0.9677	0.9677	0.9677	0.998	[[31, 1], [1, 30]]
random_forest	0.8969	0.8824	0.9677	0.9231	0.9859	[[28, 4], [1, 30]]
gradient_boosting	0.8707	0.9355	0.9355	0.9355	0.9879	[[30, 2], [2, 29]]

Selection. logistic_regression has the highest 5-fold cross-validated F1 on the training split (0.9339 vs 0.1309 for the majority baseline). Selection used training-only cross-validation, not the test set. The majority baseline never predicts high risk (precision, recall and F1 = 0; AUC 0.5). Accuracy alone would be misleading, so precision, recall, F1 and ROC-AUC are all reported.



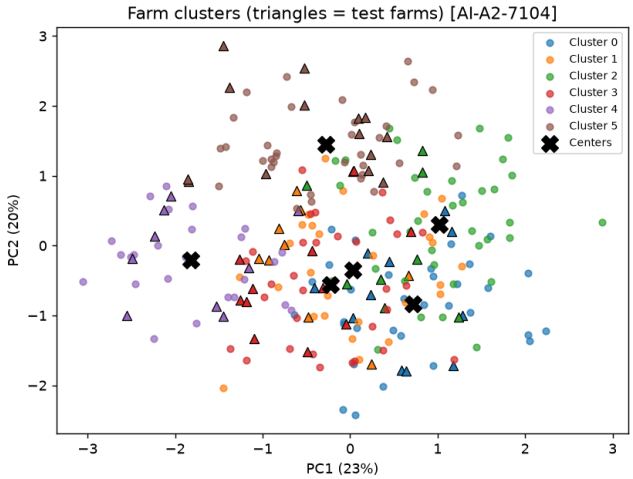
Confusion-matrix interpretation. The selected model made 1 false positive and 1 false negative among 63 farms (precision 0.9677, recall 0.9677). The classes are balanced (about 50% high risk), so the result is not driven by class imbalance. Test-set size is small, so one more error changes recall by about 3 percentage points: the metrics are indicative, not exact.

One important error (my analysis). The most serious classification error is a false negative: farm row 174 is truly high-risk (5.52 t on 1.57 ha) but the model gave it only 0.36 probability, so it predicted not high-risk. Several features of this farm point to risk: low rainfall (695 mm), very low fertilizer (14.5 kg), a hot average temperature (25.6 C, far from the best 18 C) and season B. However, it also receives cooperative support and has fairly good soil moisture (58 percent), and these pushed the probability down. A false negative is worse than a false alarm for a field officer, because a farm that needs help would be missed. For this reason, I recommend that farms with probabilities between about 0.3 and 0.7 are reviewed by a person instead of trusting the 0.5 threshold alone.

5. Clustering

K	2	3	4	5	6
Silhouette	0.155	0.1399	0.148	0.1558	0.165

Selected K = 6 (silhouette 0.165). K=6 gave the highest silhouette (0.165). A score this low (very weak structure) means the clusters are practical groupings of similar farms rather than sharply separated natural segments. Use them to organise field visits, not as hard categories. The scores for K = 2..6 are close (0.140 to 0.165), which is expected because the generator draws every feature independently and uniformly: the data has no natural clusters. I report this honestly instead of claiming strong segments.



Cluster profiles (training farms; yield and risk are shown after clustering and were not used to build it)

Cluster	n	Profile (vs overall average)	Mean yield (t)	High-risk share
0	33	low soil moisture, high fertilizer use, high temperature	11.719	0.455
1	27	low fertilizer use, high temperature, high field size	11.058	0.852
2	37	high soil moisture, high fertilizer use, high temperature	13.593	0.297
3	32	high field size, low temperature, low rainfall	13.37	0.938
4	25	low rainfall, low fertilizer use, low field size	8.129	0.4
5	34	high rainfall, low temperature, low field size	12.164	0.147

One cluster and its operational meaning (my analysis). I interpret Cluster 4 (25 farms): low rainfall (mean 715 mm), little fertilizer (mean 58 kg) and small fields (mean 1.33 ha). After clustering, this group has the lowest mean yield (8.13 t). Its high-risk share is only 40 percent, because risk is measured per hectare, so these farms produce little mostly because they are small, not only because they are inefficient. A field officer could visit this group to discuss fertilizer access and water management. I also note that the clusters are weak (silhouette 0.165), so this is a practical grouping, not a natural segment.

6. API and testing

Endpoint	Purpose	Responses
GET /health	Liveness and model availability, includes verification code	200 ok; 503 if models missing
GET /model-info	Student id, seed, AI-A2-7104, dataset fingerprint, selected models, metrics, cluster profiles	200
POST /predict	Typed JSON body (7 fields, strict types, ranges); returns yield estimate, risk, probability, cluster, verification code	200; 422 validation error; 503

Valid request (samples/valid.json) and response (HTTP 200)


```
{
 "yield_estimate_tons": 12.5071,
 "high_risk_prediction": 0,
 "high_risk_label": "not high risk",
 "high_risk_probability": 0.0709,
 "cluster": 5,
 "cluster_profile": "Cluster 5: high rainfall, low temperature, low field size",
 "verification_code": "AI-A2-7104"
}
```


Invalid request (samples/invalid.json) and response (HTTP 422)


```
{
 "error": "validation_error",
 "detail": [
  {
   "field": "rainfall_mm",
   "message": "Input should be less than or equal to 1400",
   "type": "less_than_equal"
  },
  {
   "field": "avg_temp_c",
   "message": "Input should be a valid number",
   "type": "float_type"
  },
  {
   "field": "field_size_ha",
   "message": "Field required",
   "type": "missing"
  },
  {
   "field": "season",
   "message": "Input should be 'A' or 'B'",
   "type": "literal_error"
  }
 ],
 "verification_code": "AI-A2-7104"
}
```


Failure behaviour. Missing field, wrong type, out-of-range value, unknown category, extra field and malformed JSON all return 422 with a compact list of field/message pairs; submitted values are not echoed back. predict.py prints the same error list and exits with code 2. **Tests:** 11 pytest tests pass (`pytest -q`): valid inference, missing field, wrong type, range error, unknown category / extra field, reproducibility (two full runs give identical fingerprint and metrics), dataset signature (254 rows, 13 missing, 3 duplicates), wrong seed rejection, leakage guard, API health/model-info/predict, API 422 errors.

7. Model card and reflection

Item	Statement
Intended use	Classroom demonstration of a decision-support tool: rough harvest estimate, high-risk flag and farm grouping to help a field officer prioritise visits.
Data	Fully SYNTHETIC (seed 7104, 254 raw / 251 clean rows). It reflects the assignment formula, not real Musanze farms.
Performance	Regression R2 0.8586, MAE 0.852 t; classification F1 0.9677, ROC-AUC 0.998; clusters weak (silhouette 0.165). Test set has only 63 farms.
Limitations	Synthetic uniform features and an almost linear target make results look better than real life; no temporal validation; small test set; weak clusters; inputs outside the trained ranges are rejected.
Risks	Over-trusting a prediction; wrong ranking of farms; unfair treatment if used for support decisions; stale model if climate or practices change.
Prohibited use	Deciding access to loans, insurance, subsidies or any public service; replacing an agronomist; use on real personal data without consent and a new validation.
Human oversight	A field officer or agronomist must review every output and may override it. Borderline probabilities (about 0.3-0.7) need a site visit.

Reflection - what I would improve with real data

With real data I would first check data quality with the cooperative, because spreadsheets can have wrong units and mixed formats. I would add features such as potato variety, planting date, pests and soil tests, and I would test the model on a later season instead of a random split. I would also calibrate the risk probabilities and compare errors across farmer groups to check fairness, and I would ask field officers if the risk flags are useful in practice.

Reproduce: `python run_all.py --student-id ug2527104@ines.ac.rw --seed 7104` | verification code AI-A2-7104 | SHA-256 73f47f269966ba51...